Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S004**
Sap ID: 60018250070	**Topic: Data Structure**
Branch: Artificial Intelligence & Data Science	

Experiment No. 2

Aim: To develop a program using Stack to convert an Infix expression into a Postfix expression.

Theory:

Definition:

Infix notation is the standard way of writing mathematical expressions where an operator is placed between its two operands, such as A + B. Postfix notation, also called Reverse Polish Notation, places the operator after both its operands, such as A B +. Computers evaluate postfix expressions more efficiently because they can be processed from left to right without needing parentheses or precedence checks at the evaluation stage. A stack is used during conversion to hold operators temporarily until their correct position in the postfix expression is determined based on precedence and associativity.

Operations:

Infix to Postfix Conversion: It is used to convert an infix expression into its equivalent postfix form using a stack.

Pseudocode:

```
BEGIN POSTFIX(c)

   // Check if c is an operator or parenthesis
   IF c == '(' OR c == ')' OR c == '+' OR c == '-'
     OR c == '*' OR c == '/' OR c == '^' THEN

      // Case 1: Opening parenthesis
      IF c == '(' THEN
         CALL PUSH(c)

      // Case 2: Closing parenthesis
      ELSE IF c == ')' THEN
         WHILE stack[tos] != '(' DO
            CALL POP()
         END WHILE
         tos = tos - 1        // Discard '(' from stack

      // Case 3: High precedence operators * / ^
      ELSE IF c == '*' OR c == '/' OR c == '^' THEN
         IF STACKTOP() == '^' THEN
            WHILE STACKTOP() == '^' DO
               CALL POP()
            END WHILE
            CALL PUSH(c)
         ELSE
            CALL PUSH(c)
         END IF
```

```
    // Case 4: Low precedence operators + -
    ELSE IF c == '+' OR c == '-' THEN
       IF STACKTOP() == '*' OR STACKTOP() == '/'
         OR STACKTOP() == '^' THEN
          WHILE STACKTOP() == '*' OR STACKTOP() == '/'
              OR STACKTOP() == '^' DO
            CALL POP()
          END WHILE
          CALL PUSH(c)
       ELSE
          CALL PUSH(c)
       END IF

    END IF

  // Case 5: Operand - directly print
  ELSE
     PRINT c
  END IF

END POSTFIX
```

Code:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
```

```
#define MAX 100
char stack[MAX];
int tos=-1;

void push(char c)
{
   if(tos == (MAX-1))
   {
      printf("Stack Overflow\n");
   }
   else
   {
      stack[++tos]=c;
   }
}

void pop()
{
   if(tos == -1)
   {
      printf("Stack Underflow\n");
   }
   else
   {
      printf("%c",stack[tos--]);
   }
}

char stackTop()
{
   return stack[tos];
}

void postfix(char c)
{
   if(c == "(" || c == ")" || c== "+" || c== "-" || c== "*" || c== "/" || c=="^")
   {
```

```
if(c == "(")
{
   push(c);
}
else if(c==")")
{
   while(stack[tos]!="(")
   {
      pop();
   }
   tos--;
}
else if(c=="*" || c=="/" || c=="^")
{
   if(stackTop()=="^")
   {
      while(stackTop()== "^")
      {
         pop();
      }
      push(c);
   }
   else
   {
      push(c);
   }
}
else if(c == "+" || c=="-")
{
   if(stackTop() == "*" || stackTop() ==" /" || stackTop() == "^")
   {
      while(stackTop()== "*" || stackTop()=="/" || stackTop() == "^")
      {
         pop();
      }
      push(c);
   }
```

```
            else
            {
               push(c);
            }
         }
      }
      else
      {
         printf("%c",c);
      }
   }

   void main()
   {
      int n,i;
      char s[100];
      clrscr();
      printf("Infix to postfix expression\n");
      printf("enter expression : \n");
      gets(s);
      n=strlen(s);
      for(i=0;i<n;i++)
      {
         postfix(s[i]);
      }
      for(i=tos;i>=0;i--)
      {
         pop();
      }
      fflush(stdin);
      getch();
   }
```

Output:

```
Infix to postfix expression
```

```
enter expression :
A+(B*C-(D/E^F)*G)*H
ABC*DEF^/G*-H*+
=== Code Execution Successful ===

Infix to postfix expression
enter expression :
A+B*(C^D-E)^(F+G*H)-I
ABCD^E-FGH*+^*+I-


=== Code Execution Successful ===

Infix to postfix expression
enter expression :
A+B*C^D-E
ABCD^*+E-


=== Code Execution Successful ===
```

Applications:

- Used inside compilers and interpreters to evaluate arithmetic expressions efficiently.
- Used in scientific calculators before computing results of complex expressions.
- Used in building expression trees for expression processing.
- Used in stack-based programming language implementations where postfix evaluation is the standard approach.

Conclusion:

This program successfully converted an infix expression to a postfix expression using a stack. The stack managed operator precedence and handled parentheses correctly throughout the conversion. Postfix expressions remove the need for parentheses and precedence handling at the evaluation stage, which makes them more efficient for machine-level computation. The program confirmed that a stack is the right data structure for implementing infix to postfix conversion.